비교 평가 · 자체 평가 결과 보고서

LangChain 버전 마이그레이션 코드 어시스턴트

항목	내용
팀	SKN35 3rd 4Team
작성	4번 (화면 · DB · 평가)
실행	2026-10-15 11:02:11
대상	팀 저장소 dev 최신 그래프 (최상위 supervisor) · 모델 gpt-4o-mini
평가 도구	eval/run_eval.py (실행) · eval/score.py (채점, LLM 사용 안 함)
관련 문서	프로젝트 최종 보고서 (설계 · 역할별 작업 기록 · 트러블슈팅 전체) · 테스트 계획 및 결과 보고서

요약

57%
우리 챗봇 · 공통 기준 통과
(1.x로 올리기 30개)

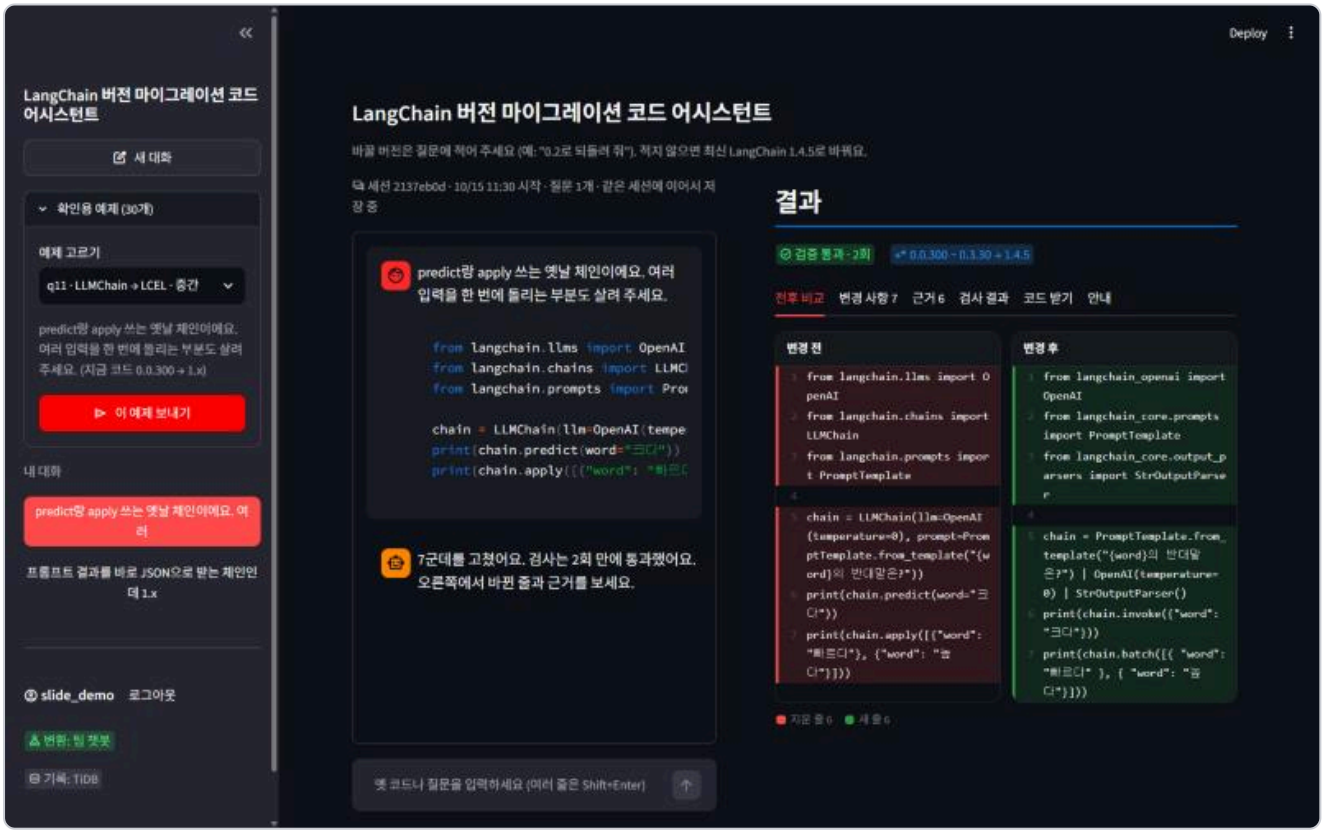
0%
일반 LLM · 공통 기준 통과
(같은 모델 gpt-4o-mini)

83.3점
자체 평가 (B등급)
목표 달성 13/21항목

53%
챗봇 검증 통과 (자체 판정)
3회차까지 누적

> 같은 모델(gpt-4o-mini)에 같은 질문 30개(1.x로 올리기)를 보냈을 때, 공통 기준 8항목을 모두 통과한 답은 우리 챗봇 57%, 일반 LLM 0%였습니다. 정답 import를 빼고 '실행은 되는 코드인가'만 보면 87% 대 0%입니다. 모델을 바꾼 것이 아니라 근거와 검사를 붙인 차이입니다.
>
> 일반 LLM은 30개 중 29개에서 옛 import를 그대로 두고 "1.x에 맞게 수정했다"고 답했습니다(버전 지체 환각). 우리 챗봇은 바뀐 줄의 93%에 근거를 붙였고, 통과하지 못한 답은 '검증 실패'로 알렸습니다.
>
> 자체 평가는 83.3점(B)입니다. 분석 · 근거 검색 · 답변 규칙 · 안정성은 98~100%로 탄탄하고, 약점(질문 분류 · 환각 · 검사기 빈틈)은 원인과 담당, 고칠 순서까지 정했습니다.

실제 화면



실제 화면 (2026-10-15) · 확인용 예제 q11(LLMChain → LCEL)을 보내 2회차에 검증 통과 · 0.0.300 ~ 0.3.30 → 1.4.5 · 바뀐 줄을 빨강 · 초록으로 비교

강점 (숫자로 확인된 것)

강점	근거
같은 모델로 0% → 57%	공통 기준 8항목 모두 통과: 챗봇 57% · 일반 LLM 0%
실행되는 코드	정답 import 제외 '실행 기준' 87% vs 0%
옛 import를 확실히 걷어냄	옛 import 없음 93% vs 3%, 새 import가 1.x에 실제로 있음 90% vs 3%
환각을 '통과'로 내보내지 않음	기록 없는 변경(환각)은 검사기가 모두 막음. 실패한 답은 화면에 '검증 실패'와 이유
근거가 붙은 답	바뀐 줄 103개 중 96개(93%)에 근거 ID
근거 검색 · 분석	대응표의 옛 import 99%(80/81)를 근거로 가져옴, 목표 버전 읽기 100%
쉬운 · 경로형 문제는 완벽	쉬움 100%, import 경로 · 문서 로더 유형 100% (일반 LLM 0%)
답변 규칙 · 안정성	출처 표기 100%, 50개 모두 오류 없이 응답, 변환 중앙값 11.6초

1. 평가 설계

1-1. 두 가지 평가

	비교 평가	자체 평가
질문	일반 LLM보다 나은가?	우리가 정한 규칙대로 동작하는가?
대상	우리 챗봇 vs 일반 LLM (같은 질문, 같은 모델)	우리 챗봇만
기준	공통 채점 8항목 (두 답에 같은 잣대)	팀 프롬프트 · 코드에 적힌 규칙 21항목 (100점)
쓰는 곳	발표 결과 슬라이드, README 결과	다음 개선 순서 정하기, 담당별 전달

1-2. 실험 조건

항목	내용
질문	eval/questions.json 36개 (1.x로 올리기 30 + 버전 오가기 6) · 분류 확인 질문 eval/route_probes.json 14개
입력	요청 문장 + 코드 블록. 두 시스템에 글자 하나 다르지 않게 같은 글
일반 LLM	gpt-4o-mini, temperature 0, 시스템 프롬프트 없음 (ChatGPT에 붙여 넣는 상황)
우리 챗봇	팀 그래프 그대로 (분류 → 분석 → 근거 검색 → 고쳐 쓰기 → 검사, 최대 3회). 화면과 같은 호출 방식
채점	LLM 없이 코드로만 채점 (사람 · LLM 판정의 주관 없음). 같은 결과를 몇 번 채점해도 같은 점수
횟수	질문마다 1회 실행 (분류 일관성만 3회)

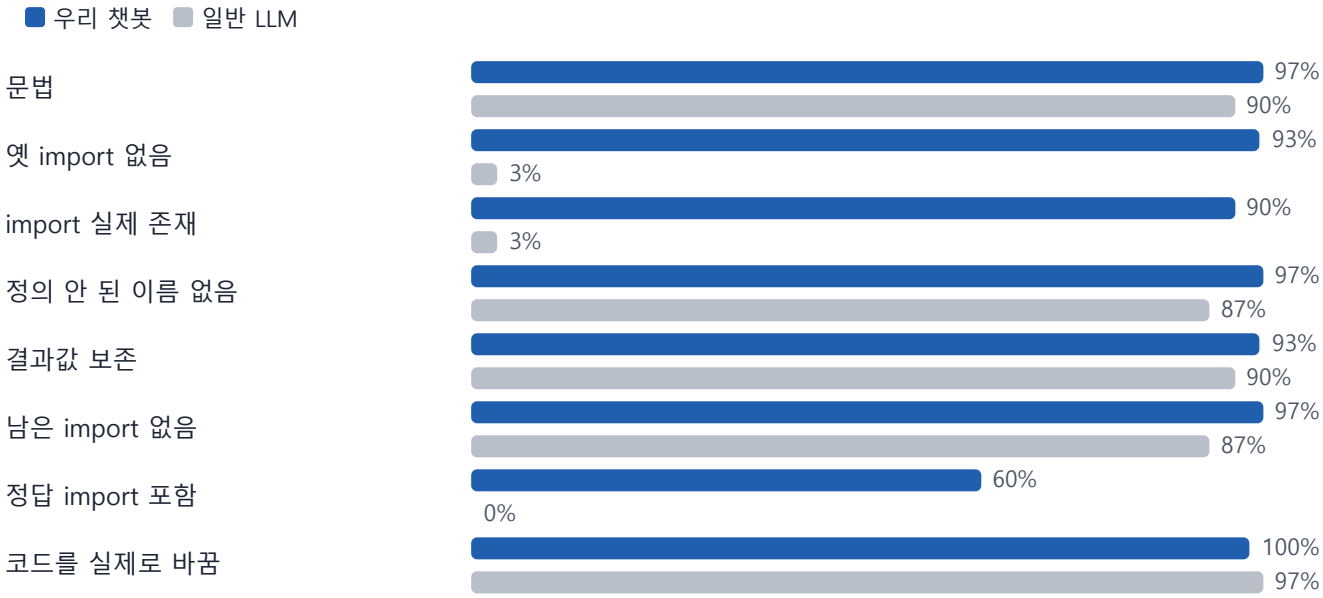
1-3. 공통 채점 8항목

일반 LLM 답에는 근거 기록이 없으므로, 근거 ID · 변경 기록처럼 우리 챗봇에만 있는 검사는 넣지 않았습니다. 8개를 모두 통과해야 '공통 기준 통과'입니다.

#	항목	통과 조건	확인 방법
1	문법	파이썬으로 읽힘	ast
2	옛 import 없음	2번 대응표의 옛 경로가 남지 않음	import_map 9,155줄
3	import 실제 존재	새 import 경로가 LangChain 1.x에 있음	2번 버전별 도구 목록 (1.4.5)
4	정의 안 된 이름 없음	원래 코드엔 없던 NameError가 생기지 않음	ast
5	결과값 보존	문자열 결과 유지, LCEL 결과를 ['text']로 꺼내지 않음, 체인 입출력 키가 맞음	verifier 8 · 10 · 11번 + 더 넓은 꺼내기 검사
6	남은 import 없음	바꾸고 안 쓰게 된 import가 남지 않음	ast
7	정답 import 포함	질문마다 사람이 정한 정답 import가 모두 들어 있음	questions.json expected_imports
8	코드를 실제로 바꿈	원문 그대로 돌려주지 않음	줄 비교

2. 비교 평가 결과

2-1. 1.x로 올리기 30개



항목	우리 챗봇	일반 LLM
공통 기준 통과 (8개 모두)	**57%**	**0%**
실행 기준 통과 (정답 import 제외 7개)	87%	0%
문법	97%	90%
옛 import 없음	93%	3%
import 실제 존재	90%	3%
정의 안 된 이름 없음	97%	87%
결과값 보존	93%	90%
남은 import 없음	97%	87%
정답 import 포함	60%	0%
코드를 실제로 바꿈	100%	97%
응답 시간 (중앙값)	11.6초	3.5초
남은 옛 메서드 호출 (경고 수, 적을수록 좋음)	2	17

2-2. 난이도 · 유형별 공통 기준 통과

난이도	챗봇	일반 LLM
쉬움	100%	0%
중간	53%	0%
어려움	25%	0%

유형 (각 3개)	챗봇	일반 LLM
LLMChain → LCEL	67%	0%
RAG	33%	0%
import 경로	100%	0%
메모리	33%	0%
문서 로더 · 분할기	100%	0%
실무 상황	67%	0%
에이전트 · 도구	0%	0%
임베딩 · 벡터스토어	67%	0%
체인 조합	33%	0%
출력 파서	67%	0%

2-3. 실패를 알려 주는가

	개수	뜻
실패했지만 '검증 실패'로 알림	8	사용자가 결과를 그대로 믿지 않게 화면에 실패 배지와 이유를 보여 줌
검증은 통과했는데 공통 채점에서 실패	5	검사기가 놓친 경우. 4장 개선 대상
질문 분류가 틀려 변환을 안 함	6	q06, q07, q09, q20, q26, q28

일반 LLM은 틀린 코드도 맞다고 생각하고 내놓습니다. 실패 여부를 알려 주는 장치가 없습니다.

2-4. 버전 오가기 (6개, 참고)

목표가 0.x인 질문은 2번의 1.x 도구 목록으로 확인할 수 없는 항목(옛 import · import 존재 · 남은 import)을 빼고 5항목으로 채점했습니다.

	우리 챗봇	일반 LLM
공통 기준 통과	0%	33%
챗봇 검증 통과	50%	-
응답 시간 (중앙값)	19.0초	3.6초

3. 자체 평가 (점검표 · 점수표)

팀이 프롬프트와 코드에 적어 둔 규칙을 실제 실행 기록으로 확인했습니다. 점수 = 배점 × 달성률. 목표는 실무에서 '믿고 쓸 수 있다'고 볼 최소 수준으로 4번이 정했습니다.

영역	점수	달성률
질문 분류	14.0 / 15	93%
분석	9.8 / 10	98%
근거 검색	9.9 / 10	99%
고쳐 쓰기	22.3 / 25	89%
검증 · 재시도	17.3 / 30	58%
답변 규칙	5.0 / 5	100%
안정성 · 속도	5.0 / 5	100%
합계	**83.3 / 100**	**B등급**

> **등급**: A 90점 이상 · B 80 · C 70 · D 70 미만
>
> **읽는 법**: '미달'인 항목이 다음에 고칠 곳입니다. 마지막 칸은 미달을 만든 질문 번호입니다.

#	영역	점검 항목	팀이 정한 근거	결과	목표	판정	점수	미달 질문
A1	질문 분류	코드 변환 요청을 migration으로 분류	classifier 프롬프트 · 분류 규칙 1, 2	84% (32/38)	100%	미달	5.1/6	q06, q07, q09, q20, q26, q28
A2	질문 분류	지식 · 일반 · 지시문 섞인 질문을 맞게 분류	classifier 프롬프트 · 분류 규칙 2~6	100% (12/12)	90%	달성	5.0/5	
A3	질문 분류	같은 질문을 3번 보내도 분류가 같음	라우터는 출력이 정확히 하나 (프롬프트 1행)	98% (49/50)	100%	미달	3.9/4	q20
B1	분석	질문에서 코드 원문을 그대로 꺼냄	analyzer 프롬프트 · old_code 규칙	97% (29/30)	95%	달성	4.8/5	q32
B2	분석	질문에 적은 목표 버전을 맞게 읽음 (없으면 최신)	analyzer target_version 규칙 · 2번 normalize_version	100% (30/30)	100%	달성	5.0/5	
C1	근거 검색	근거를 1개 이상 찾음	retriever: 대응표 정확 매칭 + 하이브리드 검색	100% (30/30)	100%	달성	4.0/4	
C2	근거 검색	대응표에 있는 옛 import는 근거로 꼭 가져옴	retriever · import_lookup	99% (80/81)	95%	달성	5.9/6	q35:langchain.agents.AgentExecutor

#	영역	점검 항목	팀이 정한 근거	결과	목표	판정	점수	미달 질문
D1	고쳐 쓰기	바꾼 줄마다 보여 준 근거 ID만 붙임	rewriter 원칙 2 · verifier 2번	87% (26/30)	95%	미달	5.2/6	q13, q14, q23, q24
D2	고쳐 쓰기	기록 없이 바뀐 줄이 없음 (환각 없음)	rewriter 원칙 1, 규칙 6 · verifier 3번	70% (21/30)	95%	미달	4.2/6	q03, q13, q14, q18, q23, q24, q25, q29 …
D3	고쳐 쓰기	결과값 · 출력 꺼내기 · 체인 키를 지킴	rewriter 규칙 3 · verifier 8, 10, 11번	100% (30/30)	95%	달성	5.0/5	
D4	고쳐 쓰기	새로 쓴 이름은 import도 추가	rewriter 규칙 4 · verifier 9번	97% (29/30)	95%	달성	3.9/4	q31
D5	고쳐 쓰기	안 쓰게 된 import는 지움	rewriter 규칙 5 · verifier 7번	100% (30/30)	95%	달성	4.0/4	
E1	검증 · 재시도	최종 검증 통과	verifier 11가지 · 최대 3회	63% (19/30)	80%	미달	6.3/10	q03, q13, q14, q18, q23, q24, q25, q29 …
E2	검증 · 재시도	1회차에 검증 통과	-	47% (14/30)	60%	미달	2.3/5	q03, q04, q13, q14, q18, q22, q23, q24 …
E3	검증 · 재시도	1회차 실패 → 다시 시도해서 통과	실패 이유 피드백 + 틀린 근거 제외	31% (5/16)	50%	미달	1.6/5	q03, q13, q14, q18, q23, q24, q25, q29 …
E4	검증 · 재시도	검증 시도가 최대 3회를 넘지 않음	MAX_ATTEMPTS = 3	100% (30/30)	100%	달성	3.0/3	
E5	검증 · 재시도	검증 통과한 코드는 공통 채점도 통과	검사기 신뢰도	58% (11/19)	100%	미달	4.1/7	q10, q15, q16, q21, q22, q32, q34, q36
F1	답변 규칙	LangChain 질문 답에 '출처' 표기	search 프롬프트 · 답변 형식	100% (5/5)	90%	달성	3.0/3	
F2	답변 규칙	한국어로 답함	search · general 프롬프트	100% (12/12)	100%	달성	2.0/2	
G1	안정성 · 속도	오류 없이 답을 돌려줌	-	100% (50/50)	100%	달성	3.0/3	
G2	안정성 · 속도	코드 변환 60초 안에 끝남	화면 기다림 한도 (4번)	100% (30/30)	90%	달성	2.0/2	

3-1. 다시 시도의 효과 (1.x로 올리기)

	검증 통과 (누적)
1회차까지	43%
2회차까지	53%
3회차까지	53%

4. 약점과 개선 방안

① ~ ④는 고칠 순서. 담당은 4번 제안이며 팀 확인 후 확정합니다.

#	무엇	근거 (이번 실행)	담당	개선안
1	코드 변환 요청이 'LangChain 질문 (search)'으로 분류됨	1.x로 올리기 30개 중 6개(q06 · q07 · q09 · q20 · q26 · q28). 코드와 함께 '에러 이유', '무엇을 설치', '어디가 바뀌었는지'를 같이 물으면 '고쳐 주세요'가 있어도 search. 3번 반복해도 같은 결과라 우연이 아니라 규칙 문제	1번	① 분류 규칙에 '코드가 있고 고치기 · 바꾸기 의도가 있으면 설명 요청이 섞여도 migration' 추가, classifier temperature 0.7 → 0 (6개가 바로 변환 경로로 → 최종 통과 약 67% 기대)
2	기록 없는 변경(환각)이 가장 흔한 실패 원인	모든 시도의 오류 중 1위 unrecorded_change 33회. 최종 실패 11건 중 9건에 포함. 1회차 실패 후 다시 시도해 통과한 비율 31%(5/16)	3번 · 1번	② rewriter 프롬프트에 '요청 코드에 없던 줄 · 문서 예시 코드를 새로 넣지 말 것' 명시, 환각을 만든 근거도 다음 시도에서 제외 (EVIDENCE_FAULT_TYPES에 unrecorded_change 추가) (검사기가 막고 있어 '통과'로 나간 환각은 없음)
3	바꾼 줄에 근거 ID를 안 붙임	missing_evidence 23회(전체 시도). 최종 실패 4건(q13 · q14 · q23 · q24)	3번	② 출력 형식에서 evidence_ids를 필수로(빈 목록이면 구조화 출력 단계에서 거절), 재시도 피드백에 '어느 근거를 붙이면 되는지' 후보 제시
4	검사기 빈틈 ①: LCEL 결과를 ['text']로 꺼내는 코드를 통과시킴	q10: result = chain.invoke(...) 다음 줄에서 result["text"] (실행하면 TypeError). 10번 검사는 chain.invoke(...)["text"]처럼 한 줄에 쓴 경우와 llm 변수로 끝나는 체인만 봄	3번	③ 결과를 변수에 담은 뒤 꺼내는 경우, 체인 끝이 ChatOpenAI(...) 직접 호출인 경우까지 검사 (eval/score.py의 lcel_subscript와 같은 방식) → ✅ 평가 직후 3번이 반영 (dev 648b794 · 0c8821e)
5	검사기 빈틈 ②: 목표 버전을 보지 않고 항상 1.x 기준으로 검사	q32: 0.x로 되돌려 달라는 요청에 1.x 전용 create_agent를 쓴 코드가 '검증 통과'. 버전 오가기 6개 중 정답 import를 맞춘 답 0개	3번 · 2번	③ verifier가 state의 target_version을 받아 2번 version_resolve(버전별 도구 목록)로 import 존재를 확인
6	최신 방식 대신 langchain_classic으로 우회	q15(RunnableBranch 대신 MultiPromptChain), q21(체크포인터 대신 ConversationBufferMemory), q22(create_agent 대신 initialize_agent). 실행은 되지만 '지원 종료 예정' 경로	2번 · 3번	④ retriever가 대안(alternative) 사용법 문서를 함께 찾도록, rewriter는 classic으로 우회하면 notes에 '최신 방식 아님 + 대안'을 반드시 적기
7	속도는 목표 안	코드 변환 중앙값 11.6초(최대도 60초 이내), 일반 LLM 3.5초. 다시 시도가 붙으면 30초 안팎	-	화면의 진행 단계 실시간 표시로 기다림 체감 줄이기 (적용 완료)

5. 프로젝트 의의

1. 모델을 바꾸지 않고 신뢰를 만든다: 같은 gpt-4o-mini가 혼자서는 0%, 근거 + 코드 검사 + 재시도를 붙이면 57%(실행 기준 87%).
2. 버전 지체 환각을 실제로 재현하고 막았다: 일반 LLM은 옛 코드를 '수정했다'고 돌려줬고(29/30), 챗봇은 검사로 걸러 실패를 알렸다.
3. 틀릴 때 틀렸다고 말하는 AI: 답마다 근거(93%)와 검사 결과를 보여 줘 사용자가 직접 확인할 수 있다.
4. 스스로 점검하는 구조: 팀 규칙 21가지를 실행 기록으로 채점하는 도구로 고칠 곳과 순서를 데이터로 정했고, 개선 후 같은 도구로 효과를 다시 잰다.
5. 확장성: 대응표 · 문서 · 검사 기준만 바꾸면 빠르게 바뀌는 다른 라이브러리(pydantic, OpenAI SDK 등)에도 같은 구조를 쓸 수 있다.

6. 한계

- 질문마다 1회 실행입니다. LLM 답은 매번 조금씩 달라서 몇 %p는 오르내릴 수 있습니다 (분류만 3회 반복).
- 채점은 코드를 실제로 실행하지 않고 정적으로 확인합니다 (문법 · import 존재 · 이름 · 결과값). 실제 실행 결과가 맞는지는 확인하지 않았습니다.
- '정답 import 포함'은 질문마다 한 가지 정답 기준이라, 다른 올바른 방법도 감점될 수 있습니다. 그래서 정답 import를 뺀 '실행 기준 통과'도 함께 표시했습니다.
- 일반 LLM은 시스템 프롬프트 없이 그대로 물었습니다 (ChatGPT에 붙여 넣는 상황). 프롬프트를 잘 쓰면 더 나아질 수 있습니다.
- 질문 30개는 팀이 직접 만든 것이고, 버전 오가기 6개는 표본이 작아 참고용입니다.
- 자체 평가의 목표치는 4번이 실무 기준으로 정한 값입니다. 팀 합의로 조정할 수 있습니다.

7. 다시 해 보기

```
python eval/run_eval.py          # 질문 50개 실행 (챗봇 · 일반 LLM · 분류 3회), 끊겨도 이어서
python eval/score.py             # 채점 → eval/results/summary.json, scorecard.md
```

부록. 질문별 결과

id	유형	난이도	챗봇	챗봇 검증	일반 LLM	챗봇이 걸린 항목
q01	import 경로	쉬움	통과	통과 · 1회	실패	
q02	import 경로	쉬움	통과	통과 · 1회	실패	
q03	import 경로	쉬움	통과	실패 · 3회	실패	
q04	문서 로더 · 분할기	쉬움	통과	통과 · 2회	실패	
q05	문서 로더 · 분할기	쉬움	통과	통과 · 1회	실패	
q06	문서 로더 · 분할기	중간	통과	변환 안 함	실패	
q07	임베딩 · 벡터스토어	쉬움	통과	변환 안 함	실패	
q08	임베딩 · 벡터스토어	중간	통과	통과 · 1회	실패	
q09	임베딩 · 벡터스토어	중간	실패	변환 안 함	실패	문법 오류
q10	LLMChain → LCEL	중간	실패	통과 · 1회	실패	output_kept: line 10 ['text']; expected_imports: langchain_core.output_parsers.StrOutputParser
q11	LLMChain → LCEL	중간	통과	통과 · 1회	실패	
q12	LLMChain → LCEL	중간	통과	통과 · 1회	실패	
q13	체인 조합	중간	통과	실패 · 3회	실패	
q14	체인 조합	어려움	실패	실패 · 3회	실패	expected_imports: langchain_core.runnables.RunnablePassthrough
q15	체인 조합	어려움	실패	통과 · 1회	실패	expected_imports: langchain_core.prompts.PromptTemplate, langchain_core.runnables.RunnableBranch
q16	RAG	중간	실패	통과 · 1회	실패	expected_imports: langchain_core.prompts.ChatPromptTemplate, langchain_core.runnables.RunnablePassthrough
q17	RAG	어려움	통과	통과 · 1회	실패	
q18	RAG	중간	실패	실패 · 3회	실패	expected_imports: langchain_core.prompts.ChatPromptTemplate

id	유형	난이도	챗봇	챗봇 검증	일반 LLM	챗봇이 걸린 항목
q19	메모리	중간	통과	통과 · 1회	실패	
q20	메모리	어려움	실패	변환 안 함	실패	imports_exist: langchain.checkpoint.memory.InMemorySaver; expected_imports: langchain_core.messages.trim_messages
q21	메모리	중간	실패	통과 · 1회	실패	expected_imports: langgraph.checkpoint.memory.InMemorySaver
q22	에이전트 · 도구	어려움	실패	통과 · 2회	실패	expected_imports: langchain.agents.create_agent, langchain_core.tools.tool
q23	에이전트 · 도구	어려움	실패	실패 · 3회	실패	expected_imports: langchain.agents.create_agent
q24	에이전트 · 도구	어려움	실패	실패 · 3회	실패	expected_imports: langchain.agents.create_agent, langgraph.checkpoint.memory.InMemorySaver
q25	출력 파서	중간	통과	실패 · 3회	실패	
q26	출력 파서	중간	실패	변환 안 함	실패	no_legacy_import: langchain.chat_models.ChatOpenAI; imports_exist: langchain.chat_models.ChatOpenAI
q27	출력 파서	쉬움	통과	통과 · 2회	실패	
q28	실무 상황	중간	통과	변환 안 함	실패	
q29	실무 상황	중간	실패	실패 · 3회	실패	expected_imports: langchain_core.caches.InMemoryCache, langchain_core.globals.set_llm_cache
q30	실무 상황	어려움	통과	통과 · 1회	실패	
q31	버전 오가기	중간	실패	실패 · 3회	실패	no_new_undefined: ChatOpenAI; expected_imports: langchain.chat_models.ChatOpenAI, langchain.text_splitter.RecursiveChara
q32	버전 오가기	어려움	실패	통과 · 2회	실패	expected_imports: langchain.chat_models.ChatOpenAI
q33	버전 오가기	중간	실패	실패 · 3회	실패	expected_imports: langchain.chat_models.ChatOpenAI
q34	버전 오가기	중간	실패	통과 · 1회	통과	expected_imports: langchain_core.prompts.PromptTemplate
q35	버전 오가기	어려움	실패	실패 · 3회	실패	expected_imports: langchain.chat_models.ChatOpenAI, langchain.agents.AgentExecutor, langchain_core.prompts.MessagesPlace
q36	버전 오가기	중간	실패	통과 · 2회	통과	expected_imports: langchain.chat_models.ChatOpenAI